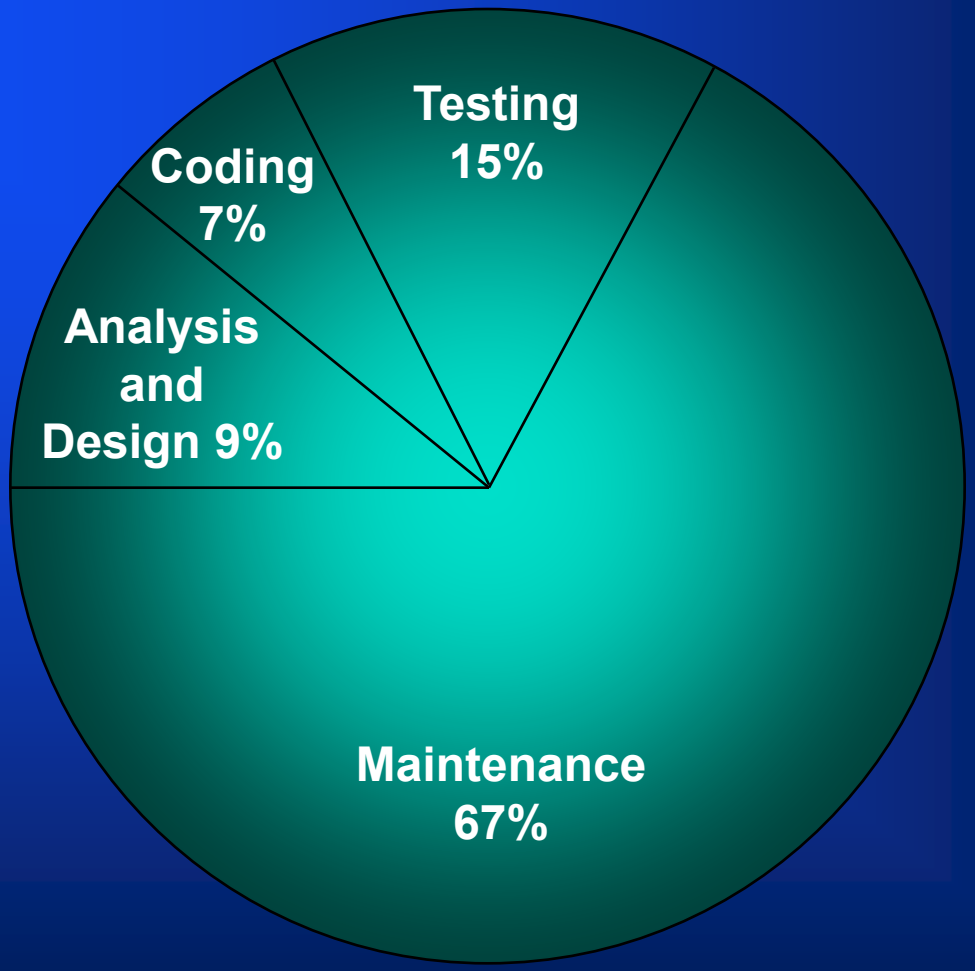
Designing Programs

COBOL

- COBOL is an acronym standing for Common Business Oriented Language.
- COBOL programs are (mostly) written for the vertical market.
- COBOL programs tend to be long lived.
- Because of this longevity ease of program maintenance is an important consideration.
- Why is program maintenance important?

Cost of a system over its entire life.



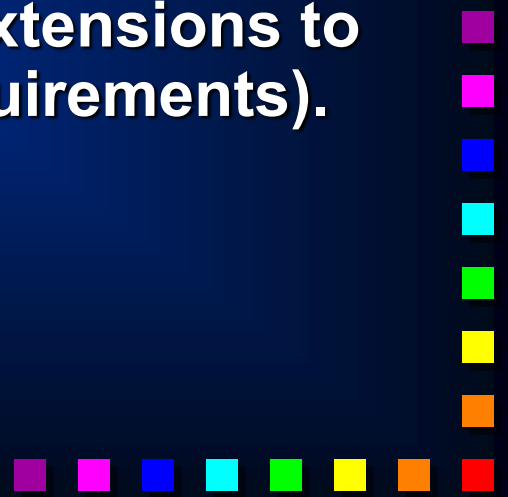
Zelkowitz
ACM 1978
p202

Maintenance Costs are only as low as this because many systems become so unmaintainable early in their lives that they have to be SCRAPPED !! :- B. Boehm

Program Maintenance.

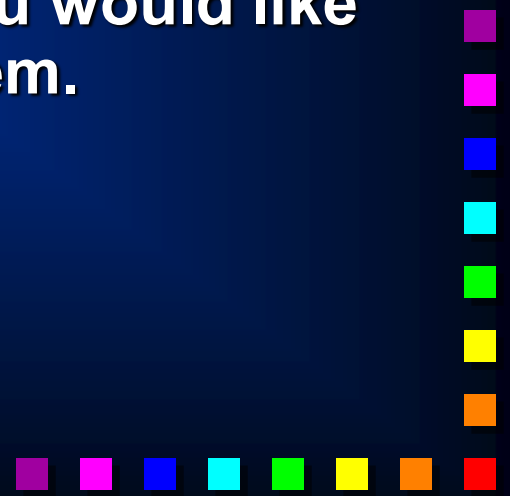
- Program maintenance is an umbrella term that covers;
  1. Changing the program to fix bugs that appear in the system.
  2. Changing the program to reflect changes in the environment.
  3. Changing the program to reflect changes in the users perception of the requirements.
  4. Changing the program to include extensions to the user requirements (i.e. new requirements).
- What do these all have in common?

CHANGING THE PROGRAM.



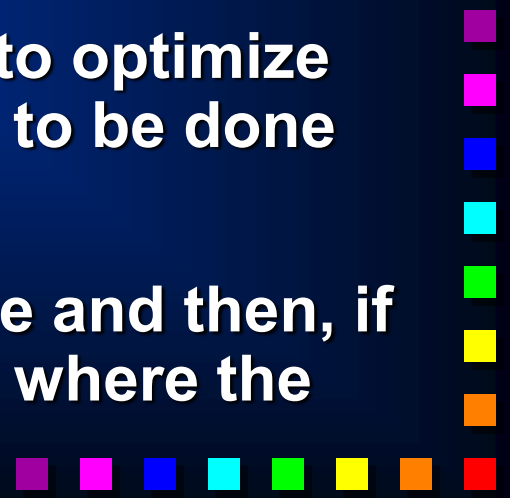
How should write your programs?

- You should write your programs with the expectation that they will have to be changed.
- This means that you should;
  - write programs that are easy to read.
  - write programs that are easy to understand.
  - write programs that are easy to change.
- You should write your programs as you would like them written if you had to maintain them.



Efficiency vs Clarity.

- Many programmers are overly concerned about making their programs as efficient as possible (in terms of the speed of execution or the amount of memory used).
- But the proper concern of a programmer, and particularly a COBOL programmer, is not this kind of efficiency, it is clarity.
- As a rule 70% of the work of the program will be done in 10% of the code.
- It is therefore a pointless exercise to try to optimize the whole program, especially if this has to be done at the expense of clarity.
- Write your program as clearly as possible and then, if its too slow, identify the 10% of the code where the work is being done and optimize it.



When shouldn't we design our programs?

- We shouldn't design our programs, when we want to create programs that do not work.
- We shouldn't design when we want to produce programs that do not solve the problem specified.
- When we want to create programs that;

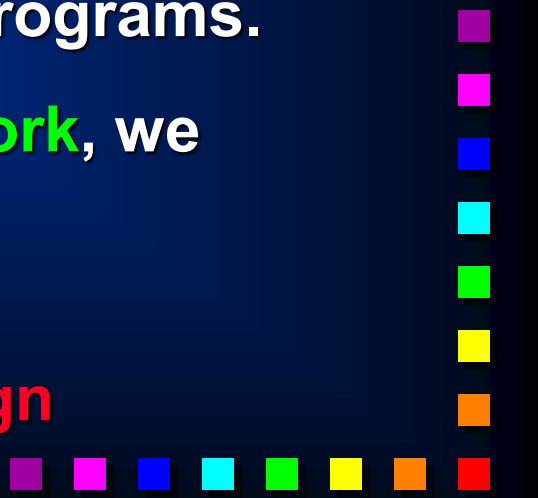
  get the wrong inputs,

  or perform the wrong transformations on them

  or produce the wrong outputs

  then we shouldn't bother to design our programs.
- But if we want to create programs that work, we cannot avoid design.
- The only question is;

  will it be a good design or a bad design

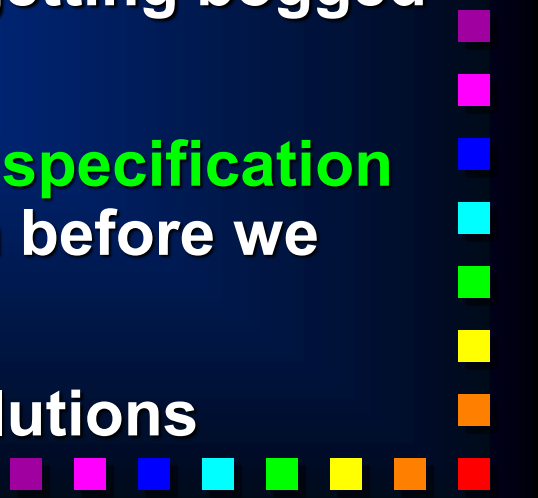


Producing a Good Design.

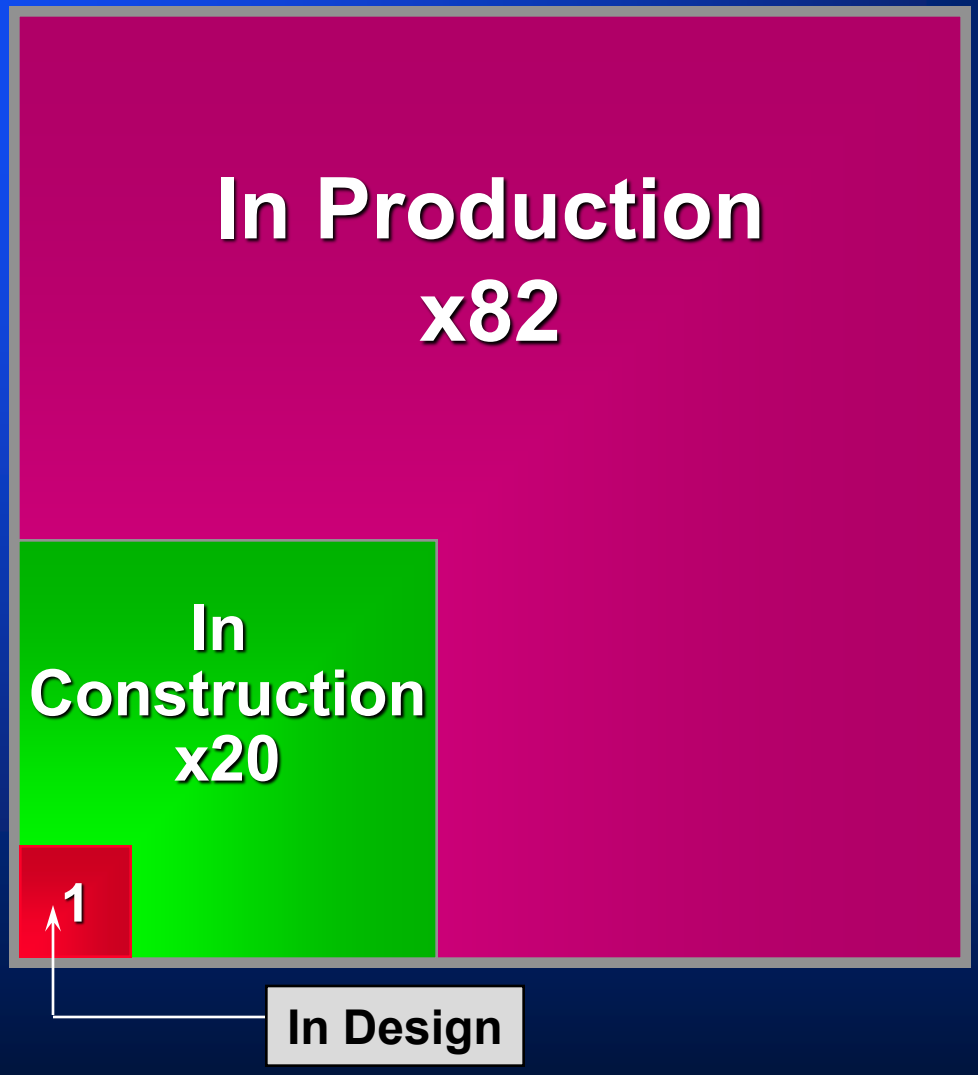
- The first step to producing a good design is to design consciously.
- Subconscious design means that design is done while constructing the program. This never leads to good results.
- Conscious design starts by separating the design task from the task of program construction.
- Design, consists of devising a solution to the problem specified.
- Construction, consists of taking the design and encoding the solution using a particular programming language.

Why separate design from construction?

- Separating program design from program construction makes both tasks easier.
- Designing before construction, allows us to plan our solution to the problem - instead of stumbling from one incorrect solution to another.
- Good program structure results from planing and design. It is unlikely to result from ad hoc tinkering.
- Designing helps us to get an overview of the problem and to think about the solution without getting bogged down by the details of construction.
- It helps us to iron out problems with the specification and to discover any bugs in our solution before we commit it to code (see next slide).
- Design allows us to develop portable solutions



Relative cost of fixing a bug.

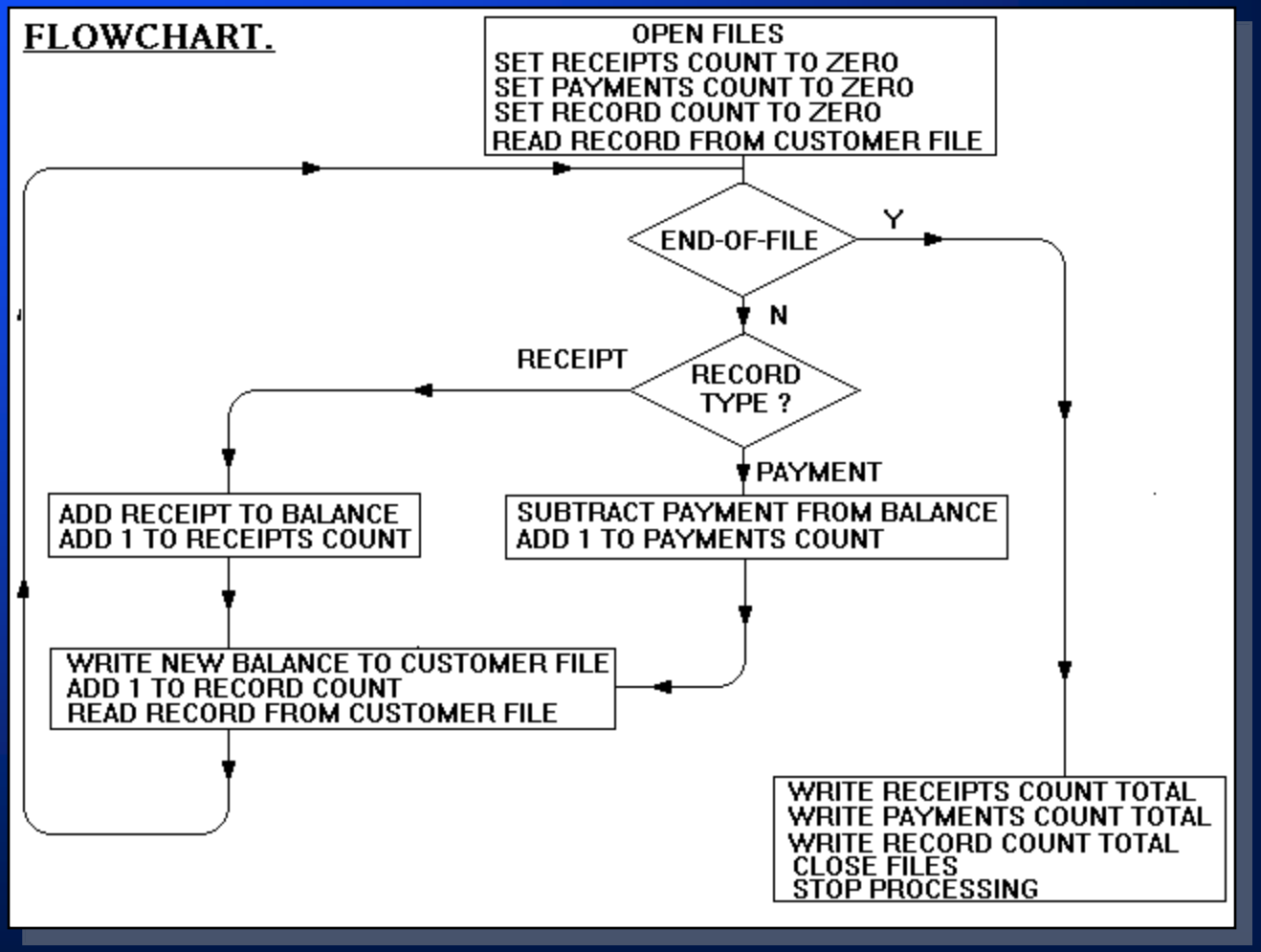


Figures from IBM in Santa Clara.

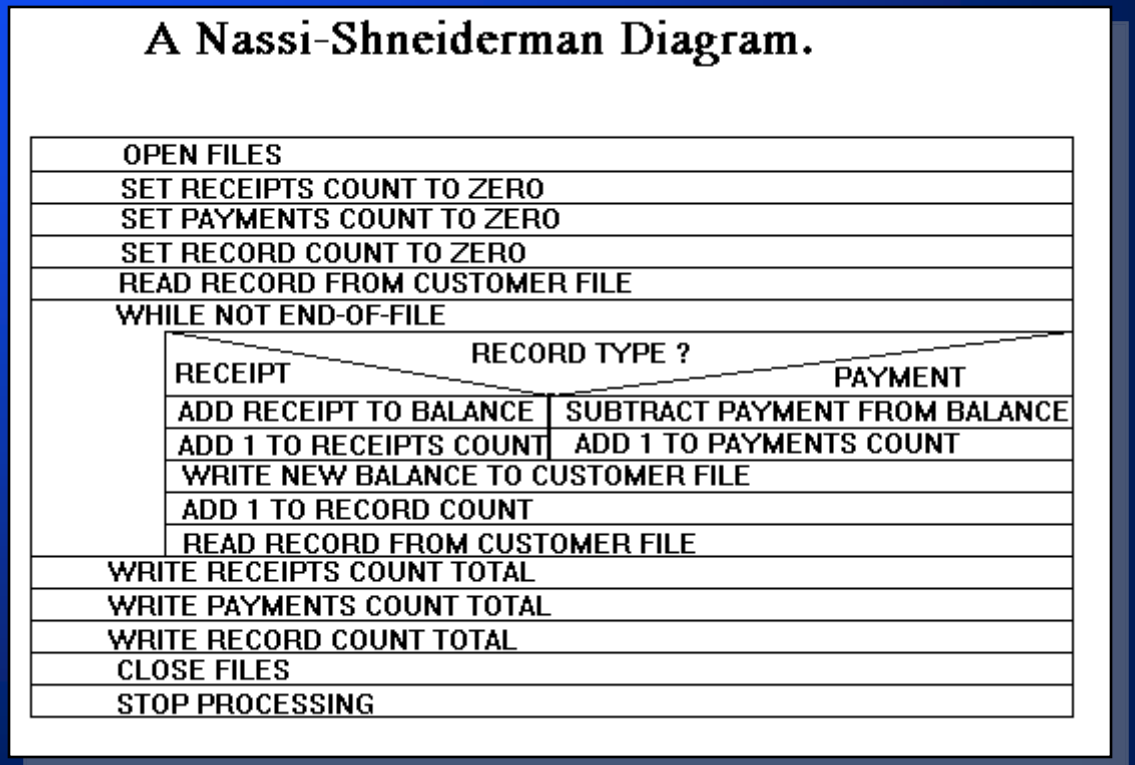
Design Notations.

- A number of notations have been suggested to assist the programmer with the task of program design.
- Some notations are textual and others graphical.
- Some notations can actually assist in the design process.
- While others merely articulate the design.

Flowcharts as design tools.



Structured Flowcharts as design tools.

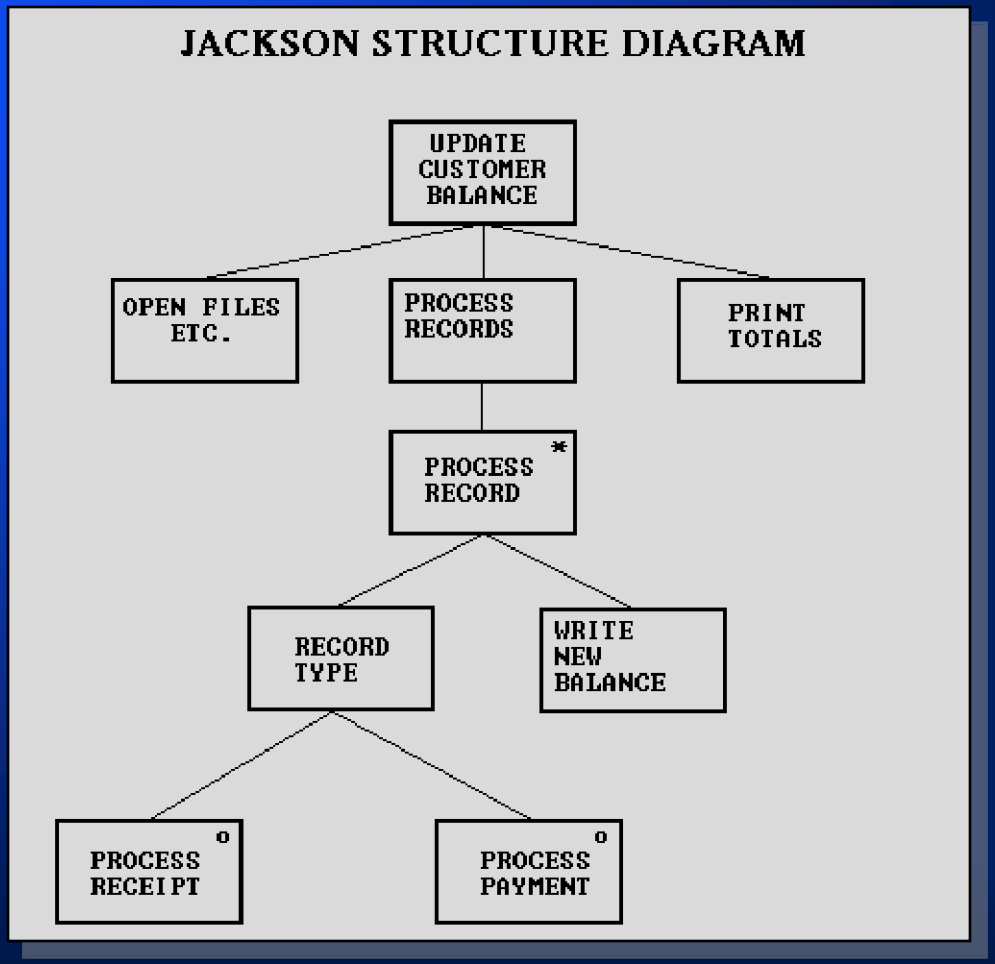


Structured English.

```
For each transaction record do the following
    IF the record is a receipt then
        add 1 to the ReceiptsCount
        add the Amount to the Balance
    otherwise
        add 1 to the PaymentsCount
        subtract the Amount from the Balance
    EndIF
    add 1 to the RecordCount
    Write the Balance to the CustomerFile

When the file has been processed
    Output    the ReceiptsCount
              the PaymentsCount
              and the RecordCount
```

The Jackson Method.



Warnier-Orr Diagrams.